

Final Project

JEONG, SEONGWON

The lid-driven cavity problem has been used a test or validation case for incompressible Navier-Stokes equation solvers. Let's consider two-dimensional unsteady flow in a square domain with Dirichlet boundary conditions on all sides, with three stationary sides and one moving side. Based on the lid velocity U and cavity height (or width) H , Reynolds number can be defined as $Re = UH/\nu$ where ν is kinematic viscosity. The velocity of the impulsively started lid is given by a step function $u_{lid} = U$ for $t \geq 0$ and $u_{lid} = 0$ for $t < 0$.

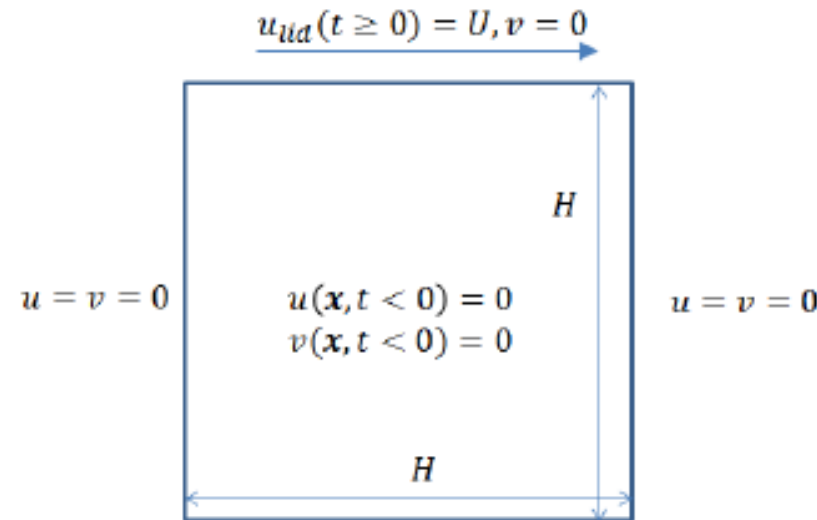


Figure 1: Schematics of flow configuration.

1. *Projection method*

Make your own fortran program to solve two-dimensional unsteady flow based on a projection method. Describe your projection method in detail. Note that you may use the explicit second-order Adams-Bashforth method for convection term and Crank-Nicolson scheme for viscous term. You need to use the second-order central difference scheme for spatial discretization in a uniform staggered grid.

1. *Projection method*

Make your own fortran program to solve two-dimensional unsteady flow based on a projection method. Describe your projection method in detail. Note that you may use the explicit second-order Adams-Bashforth method for convection term and Crank-Nicolson scheme for viscous term. You need to use the second-order central difference scheme for spatial discretization in a uniform staggered grid.

$$\frac{\partial \mathbf{u}}{\partial t} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} - (\mathbf{u} \cdot \nabla) \mathbf{u}$$

$$\frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} = -((\mathbf{u}^n \cdot \nabla) \mathbf{u}^n) + \nu \nabla^2 \mathbf{u}^n$$
$$\frac{\mathbf{u}^{n+1} - \mathbf{u}^*}{\Delta t} = -\frac{1}{\rho} \nabla p^{n+1}$$



Projection method

1. *Projection method*

Make your own fortran program to solve two-dimensional unsteady flow based on a projection method. Describe your projection method in detail. Note that you may use the explicit **second-order Adams-Bashforth method for convection term** and **Crank-Nicolson scheme for viscous term**. You need to use the second-order central difference scheme for spatial discretization in a uniform staggered grid.

$$\frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} = - \underbrace{((\mathbf{u}^n \cdot \nabla) \mathbf{u}^n)}_{\text{Convection term}} + \underbrace{\nu \nabla^2 \mathbf{u}^n}_{\text{Viscous term}}$$

$$\frac{y_{n+1} - y_n}{\Delta t} = \frac{3}{2} f_n - \frac{1}{2} f_{n-1} \quad \text{Second -order Adams Bashforth method}$$

$$\frac{y_{n+1} - y_n}{\Delta t} = \frac{1}{2} (f_{n+1} + f_n) \quad \text{Crank-Nicolson scheme}$$

$$\frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} = - \underbrace{((\mathbf{u}^n \cdot \nabla) \mathbf{u}^n)}_{\text{Convection term}} + \underbrace{\nu \nabla^2 \mathbf{u}^n}_{\text{Viscous term}}$$

$$\frac{y_{n+1} - y_n}{\Delta t} = \frac{3}{2} f_n - \frac{1}{2} f_{n-1} \quad \text{Second -order Adams Bashforth method}$$

$$\frac{y_{n+1} - y_n}{\Delta t} = \frac{1}{2} (f_{n+1} + f_n) \quad \text{Crank-Nicolson scheme}$$



$$\frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} = - \left[\frac{3}{2} (\mathbf{u}^n \cdot \nabla) \mathbf{u}^n - \frac{1}{2} (\mathbf{u}^{n-1} \cdot \nabla) \mathbf{u}^{n-1} \right] + \nu \left(\frac{1}{2} \nabla^2 \mathbf{u}^* + \frac{1}{2} \nabla^2 \mathbf{u}^n \right)$$

- Vector Form

$$\frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} = - \left[\frac{3}{2} (\mathbf{u}^n \cdot \nabla) \mathbf{u}^n - \frac{1}{2} (\mathbf{u}^{n-1} \cdot \nabla) \mathbf{u}^{n-1} \right] + \nu \left(\frac{1}{2} \nabla^2 \mathbf{u}^* + \frac{1}{2} \nabla^2 \mathbf{u}^n \right)$$

- Scalar Form

$$\begin{aligned} \frac{u_{j,i}^* - u_{j,i}^n}{\Delta t} = & - \left[\frac{3}{2} \left(u_{j,i}^n \frac{u_{j+1,i}^n - u_{j-1,i}^n}{2\Delta y} + v_{j,i}^n \frac{u_{j,i+1}^n - u_{j,i-1}^n}{2\Delta x} \right) \right. \\ & \left. - \frac{1}{2} \left(u_{j,i}^{n-1} \frac{u_{j+1,i}^{n-1} - u_{j-1,i}^{n-1}}{2\Delta y} + v_{j,i}^{n-1} \frac{u_{j,i+1}^{n-1} - u_{j,i-1}^{n-1}}{2\Delta x} \right) \right] \\ & + \nu \left[\frac{1}{2} \left(\frac{u_{j+1,i}^* - 2u_{j,i}^* + u_{j-1,i}^*}{(\Delta y)^2} + \frac{u_{j,i+1}^* - 2u_{j,i}^* + u_{j,i-1}^*}{(\Delta x)^2} \right) \right. \\ & \left. + \frac{1}{2} \left(\frac{u_{j+1,i}^n - 2u_{j,i}^n + u_{j-1,i}^n}{(\Delta y)^2} + \frac{u_{j,i+1}^n - 2u_{j,i}^n + u_{j,i-1}^n}{(\Delta x)^2} \right) \right] \end{aligned}$$

$$\frac{\mathbf{u}^{n+1} - \mathbf{u}^*}{\Delta t} = -\frac{1}{\rho} \nabla p^{n+1}$$

Helmholtz decomposition

$$\mathbf{v} = \mathbf{w} + \nabla \phi \quad \Rightarrow \quad \mathbf{u}^* = \mathbf{u}^{n+1} + \nabla \phi$$

$$\nabla \cdot \mathbf{u}^* = \nabla \cdot (\mathbf{u}^{n+1} + \nabla \phi) = 0 + \nabla^2 \phi$$



$$\nabla \cdot \mathbf{u}^{n+1} = 0$$

$$\nabla^2 \phi = \nabla \cdot \mathbf{u}^*$$

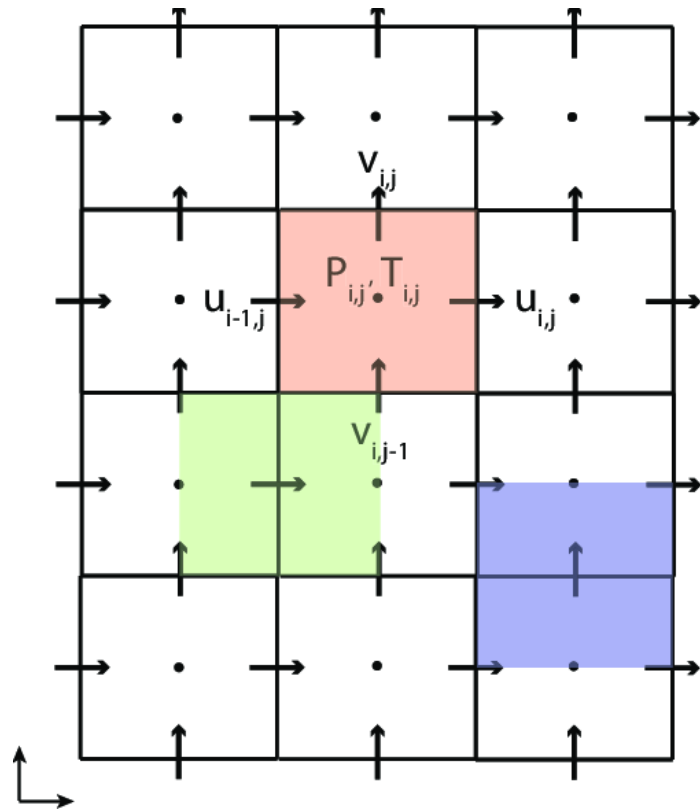
Poisson Equation



Successive Over-Relaxation(SOR)

1. *Projection method*

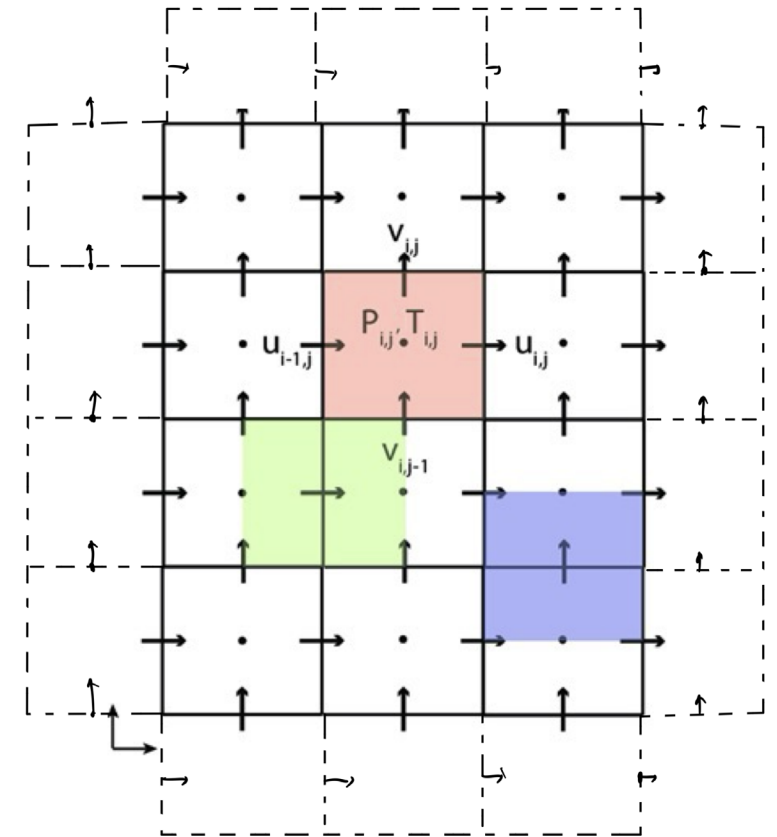
Make your own fortran program to solve two-dimensional unsteady flow based on a projection method. Describe your projection method in detail. Note that you may use the explicit second-order Adams-Bashforth method for convection term and Crank-Nicolson scheme for viscous term. You need to use the second-order central difference scheme for spatial discretization in a **uniform staggered grid**.



Boundary condition



Ghost Cell !!!




```

# initial conditions

# Space
H = 1
dx = H/128 # grid spacing
nx = int(H / dx)+1 # number of grid points in x-direction

print(nx)

# Time
dt = 0.01 # time step

# Physical parameters
nu = 1/100 # kinematic viscosity
beta = nu * dt / (2*dx**2)

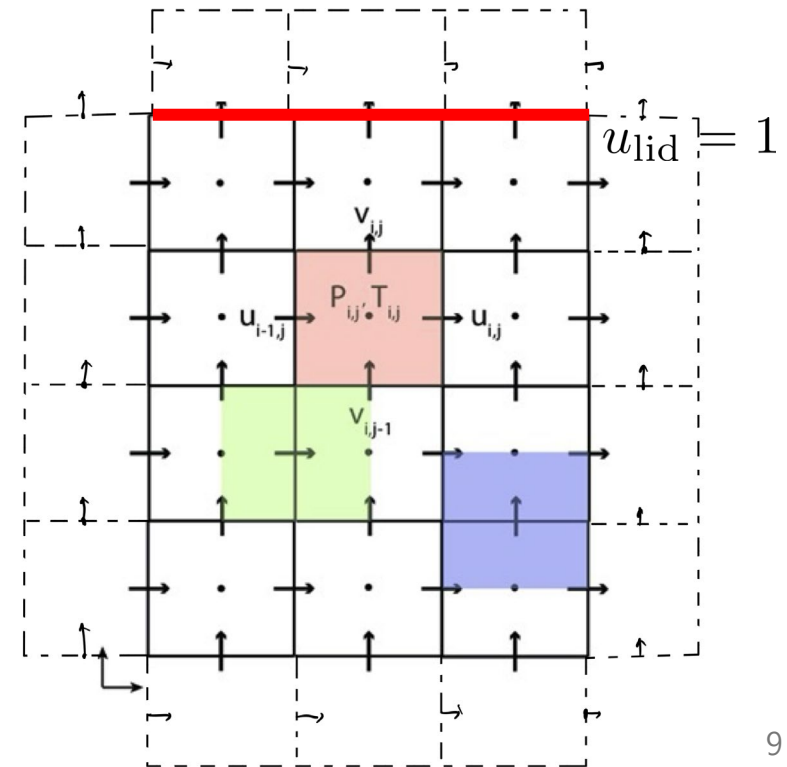
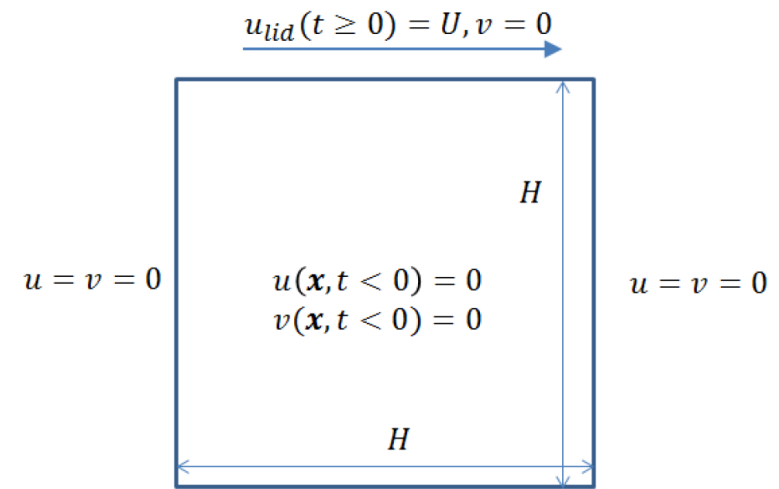
# Initial conditions
u_lid = 1
u_field = np.zeros((nx+1, nx))
v_field = np.zeros((nx, nx+1))

u_field[-1, :] = 2*u_lid # Set the lid velocity

u_field_old = np.zeros((nx+1, nx))
v_field_old = np.zeros((nx, nx+1))

u_field_list = [u_field_old, u_field]
v_field_list = [v_field_old, v_field]

```



$$\begin{aligned}
\frac{u_{j,i}^* - u_{j,i}^n}{\Delta t} = & - \left[\frac{3}{2} \left(u_{j,i}^n \frac{u_{j+1,i}^n - u_{j-1,i}^n}{2\Delta y} + v_{j,i}^n \frac{u_{j,i+1}^n - u_{j,i-1}^n}{2\Delta x} \right) \right. \\
& - \frac{1}{2} \left(u_{j,i}^{n-1} \frac{u_{j+1,i}^{n-1} - u_{j-1,i}^{n-1}}{2\Delta y} + v_{j,i}^{n-1} \frac{u_{j,i+1}^{n-1} - u_{j,i-1}^{n-1}}{2\Delta x} \right) \Big] \\
& + \nu \left[\frac{1}{2} \left(\frac{u_{j+1,i}^* - 2u_{j,i}^* + u_{j-1,i}^*}{(\Delta y)^2} + \frac{u_{j,i+1}^* - 2u_{j,i}^* + u_{j,i-1}^*}{(\Delta x)^2} \right) \right. \\
& \left. + \frac{1}{2} \left(\frac{u_{j+1,i}^n - 2u_{j,i}^n + u_{j-1,i}^n}{(\Delta y)^2} + \frac{u_{j,i+1}^n - 2u_{j,i}^n + u_{j,i-1}^n}{(\Delta x)^2} \right) \right]
\end{aligned}$$

$$\begin{aligned}
& \frac{u_{j,i}^*}{\Delta t} - \frac{\nu}{2} \left(\frac{u_{j+1,i}^* - 2u_{j,i}^* + u_{j-1,i}^*}{(\Delta y)^2} + \frac{u_{j,i+1}^* - 2u_{j,i}^* + u_{j,i-1}^*}{(\Delta x)^2} \right) \\
&= \frac{u_{j,i}^n}{\Delta t} - \left[\frac{3}{2} \left(u_{j,i}^n \frac{u_{j+1,i}^n - u_{j-1,i}^n}{2\Delta y} + v_{j,i}^n \frac{u_{j,i+1}^n - u_{j,i-1}^n}{2\Delta x} \right) \right. \\
&\quad \left. - \frac{1}{2} \left(u_{j,i}^{n-1} \frac{u_{j+1,i}^{n-1} - u_{j-1,i}^{n-1}}{2\Delta y} + v_{j,i}^{n-1} \frac{u_{j,i+1}^{n-1} - u_{j,i-1}^{n-1}}{2\Delta x} \right) \right] \\
&\quad + \frac{\nu}{2} \left(\frac{u_{j+1,i}^n - 2u_{j,i}^n + u_{j-1,i}^n}{(\Delta y)^2} + \frac{u_{j,i+1}^n - 2u_{j,i}^n + u_{j,i-1}^n}{(\Delta x)^2} \right)
\end{aligned}$$

$$\begin{aligned}
\frac{u_{j,i}^*}{\Delta t} &= \frac{\nu}{2} \left(\frac{u_{j+1,i}^* - 2u_{j,i}^* + u_{j-1,i}^*}{(\Delta y)^2} + \frac{u_{j,i+1}^* - 2u_{j,i}^* + u_{j,i-1}^*}{(\Delta x)^2} \right) \\
&= \frac{u_{j,i}^n}{\Delta t} - \left[\frac{3}{2} \left(u_{j,i}^n \frac{u_{j+1,i}^n - u_{j-1,i}^n}{2\Delta y} + v_{j,i}^n \frac{u_{j,i+1}^n - u_{j,i-1}^n}{2\Delta x} \right) \right. \\
&\quad \left. - \frac{1}{2} \left(u_{j,i}^{n-1} \frac{u_{j+1,i}^{n-1} - u_{j-1,i}^{n-1}}{2\Delta y} + v_{j,i}^{n-1} \frac{u_{j,i+1}^{n-1} - u_{j,i-1}^{n-1}}{2\Delta x} \right) \right] \\
&\quad + \frac{\nu}{2} \left(\frac{u_{j+1,i}^n - 2u_{j,i}^n + u_{j-1,i}^n}{(\Delta y)^2} + \frac{u_{j,i+1}^n - 2u_{j,i}^n + u_{j,i-1}^n}{(\Delta x)^2} \right)
\end{aligned}$$

$\beta = \frac{\nu \Delta t}{2 \Delta x}$

$$\begin{pmatrix} 1+2\beta & -\beta & 0 & \cdots & 0 \\ -\beta & 1+2\beta & -\beta & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & -\beta & 1+2\beta & -\beta \\ 0 & \cdots & 0 & -\beta & 1+2\beta \end{pmatrix} \begin{pmatrix} u_2^{n+1} \\ u_3^{n+1} \\ \vdots \\ u_{i-2}^{n+1} \\ u_{i-1}^{n+1} \end{pmatrix} = \begin{pmatrix} \beta u_3^n + (1-2\beta)u_2^n + \beta u_1^n \\ \beta u_4^n + (1-2\beta)u_3^n + \beta u_2^n \\ \vdots \\ \beta u_i^n + (1-2\beta)u_{i-1}^n + \beta u_{i-2}^n \end{pmatrix} + \begin{pmatrix} \beta u_1^{n+1} \\ 0 \\ \vdots \\ 0 \\ \beta u_i^{n+1} \end{pmatrix}$$

```
v_bar = (v_field_list[-1][:,-1]+v_field_list[-1][1:,-1]+v_field_list[-1][:,1:]+v_field_list[-1][1:,1:])/4
v_bar_old = (v_field_list[-2][:,-1]+v_field_list[-2][1:,-1]+v_field_list[-2][:,1:]+v_field_list[-2][1:,1:])/4
```

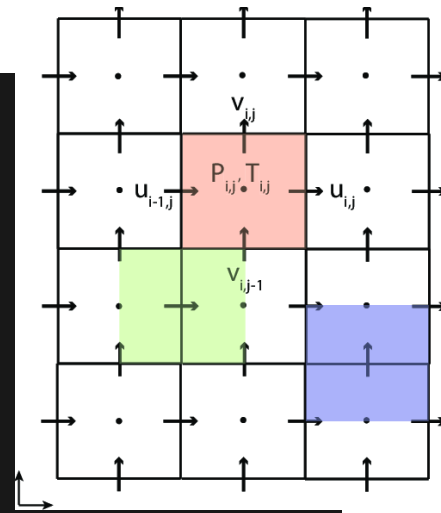
u* 구하기

```
S = -3/2 * (u_field_list[-1][1:-1, 1:-1] * (u_field_list[-1][1:-1, 2:] - u_field_list[-1][1:-1, :-2]) / (2*dx) +
            v_bar[:, 1:-1] * (u_field_list[-1][2:,1:-1] - u_field_list[-1][:,-2,1:-1]) / (2*dx)) \
+ 1/2 * (u_field_list[-2][1:-1, 1:-1]*(u_field_list[-2][1:-1, 2:] - u_field_list[-2][1:-1, :-2]) / (2*dx) +
            v_bar_old[:, 1:-1] * (u_field_list[-2][2:,1:-1] - u_field_list[-2][:,-2,1:-1]) / (2*dx))
```

```
Ix_Lx = ((Ix + beta * Lx) @ u_field_list[i][1:-1, 1:-1].T).T
Ix_Lx[:,0] = Ix_Lx[:,0] + beta * (u_field_list[i][1:-1, 0])
Ix_Lx[:,-1] = Ix_Lx[:,-1] + beta * (u_field_list[i][1:-1, -1])
```

```
R = dt*S + (Iy + beta * Ly) @ Ix_Lx
R[0,:] = R[0,:] + beta * (beta * u_field_list[i][0, :-2] + (1-2*beta) * u_field_list[i][0, 1:-1] + beta * u_field_list[i][0, 2:])
R[-1,:] = R[-1,:] + beta * (beta * u_field_list[i][-1, :-2] + (1-2*beta) * u_field_list[i][-1, 1:-1] + beta * u_field_list[i][-1, 2:])

R[0,:] = R[0,:] + beta * (-beta * u_field_list[i][0, :-2] + (1+2*beta) * u_field_list[i][0, 1:-1] - beta * u_field_list[i][0, 2:]) # u_
R[-1,:] = R[-1,:] + beta * (-beta * u_field_list[i][-1, :-2] + (1+2*beta) * u_field_list[i][-1, 1:-1] - beta * u_field_list[i][-1, 2:])
```



$$\begin{pmatrix} 1+2\beta & -\beta & 0 & \cdots & 0 \\ -\beta & 1+2\beta & -\beta & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & -\beta & 1+2\beta & -\beta \\ 0 & \cdots & 0 & -\beta & 1+2\beta \end{pmatrix} \begin{pmatrix} u_2^{n+1} \\ u_3^{n+1} \\ \vdots \\ u_{i-2}^{n+1} \\ u_{i-1}^{n+1} \end{pmatrix} = \begin{pmatrix} \beta u_3^n + (1-2\beta)u_2^n + \beta u_1^n \\ \beta u_4^n + (1-2\beta)u_3^n + \beta u_2^n \\ \vdots \\ \beta u_i^n + (1-2\beta)u_{i-1}^n + \beta u_{i-2}^n \end{pmatrix} + \begin{pmatrix} \beta u_1^{n+1} \\ 0 \\ \vdots \\ 0 \\ \beta u_i^{n+1} \end{pmatrix}$$

```
Lx = np.zeros((nx-2, nx-2))
Ix = np.eye(nx-2)

for i in range(nx-2):
    Lx[i, i] = -2

    if i == 0:
        Lx[i, i+1] = 1
    elif i == nx-3:
        Lx[i, i-1] = 1
    else:
        Lx[i, i-1] = 1
        Lx[i, i+1] = 1
Ax = (Ix - beta * Lx)
```

```
Ly = np.zeros((nx-1, nx-1))
Iy = np.eye(nx-1)

for i in range(nx-1):
    Ly[i, i] = -2

    if i == 0:
        Ly[i, i+1] = 1
    elif i == nx-2:
        Ly[i, i-1] = 1
    else:
        Ly[i, i-1] = 1
        Ly[i, i+1] = 1
Ay = (Iy - beta * Ly)
```

```

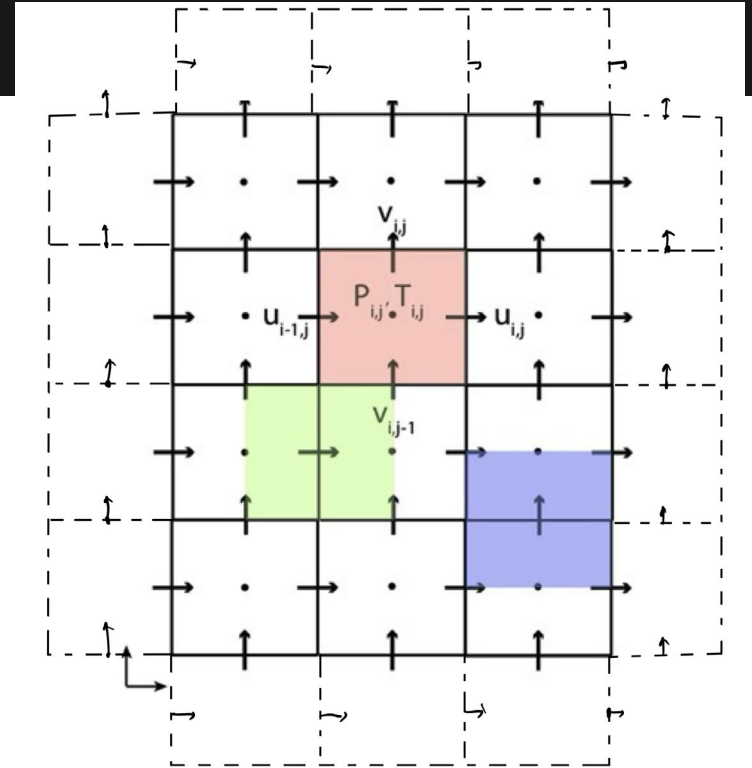
psi = np.linalg.solve(Ay, R)
psi[:,0] = psi[:,0] + beta * (u_field_list[i][1:-1, 0])
psi[:, -1] = psi[:, -1] + beta * (u_field_list[i][1:-1, -1])

u_trash = psi @ np.linalg.inv((Ix - beta * Lx).T)

u_sta = np.zeros((nx+1, nx))
u_sta[1:-1, 1:-1] = u_trash

# boundary conditions
u_sta[0,:] = -u_sta[1,:] # bottom wall
u_sta[-1,:] = 2-u_sta[-2,:] # top wall

```



v* is obtained in the same way as u*

```
# v* 구하기
u_bar = (u_field_list[-1][:,-1]+u_field_list[-1][1:,-1]+u_field_list[-1][:,-1,1]+u_field_list[-1][1:,1:])/4
u_bar_old = (u_field_list[-2][:,-1]+u_field_list[-2][1:,-1]+u_field_list[-2][:,-1,1]+u_field_list[-2][1:,1:])/4

S = -3/2 * (u_bar[1:-1, :] * (v_field_list[-1][1:-1, 2:] - v_field_list[-1][1:-1, :-2]) / (2*dx) +
            v_field_list[-1][1:-1, 1:-1] * (v_field_list[-1][2:,1:-1] - v_field_list[-1][:-2,1:-1]) / (2*dx)) \
+ 1/2 * (u_bar_old[1:-1, :]*(v_field_list[-2][1:-1, 2:] - v_field_list[-2][1:-1, :-2]) / (2*dx) +
            v_field_list[-2][1:-1, 1:-1] * (v_field_list[-2][2:,1:-1] - v_field_list[-2][:-2,1:-1]) / (2*dx))

Iy_Ly = ((Iy + beta * Ly) @ v_field_list[-1][1:-1, 1:-1].T).T
Iy_Ly[:,0] = Iy_Ly[:,0] + beta * (v_field_list[-1][1:-1, 0])
Iy_Ly[:, -1] = Iy_Ly[:, -1] + beta * (v_field_list[-1][1:-1, -1])
R = dt*S + (Ix + beta * Lx) @ Iy_Ly
R[0,:] = R[0,:] + beta * (beta * v_field_list[-1][0, :-2] + (1-2*beta) * v_field_list[-1][0, 1:-1] + beta * v_field_list[-1][0, 2:])
R[-1,:] = R[-1,:] + beta * (beta * v_field_list[-1][-1, :-2] + (1-2*beta) * v_field_list[-1][-1, 1:-1] + beta * v_field_list[-1][-1, 2:])

R[0,:] = R[0,:] + beta * (-beta * v_field_list[-1][0, :-2] + (1+2*beta) * v_field_list[-1][0, 1:-1] - beta * v_field_list[-1][0, 2:]) # u_
R[-1,:] = R[-1,:] + beta * (-beta * v_field_list[-1][-1, :-2] + (1+2*beta) * v_field_list[-1][-1, 1:-1] - beta * v_field_list[-1][-1, 2:])

psi = np.linalg.solve(Ax, R)
psi[:,0] = psi[:,0] + beta * (v_field_list[-1][1:-1, 0])
psi[:, -1] = psi[:, -1] + beta * (v_field_list[-1][1:-1, -1])

v_trash = psi @ np.linalg.inv((Iy - beta * Ly).T)

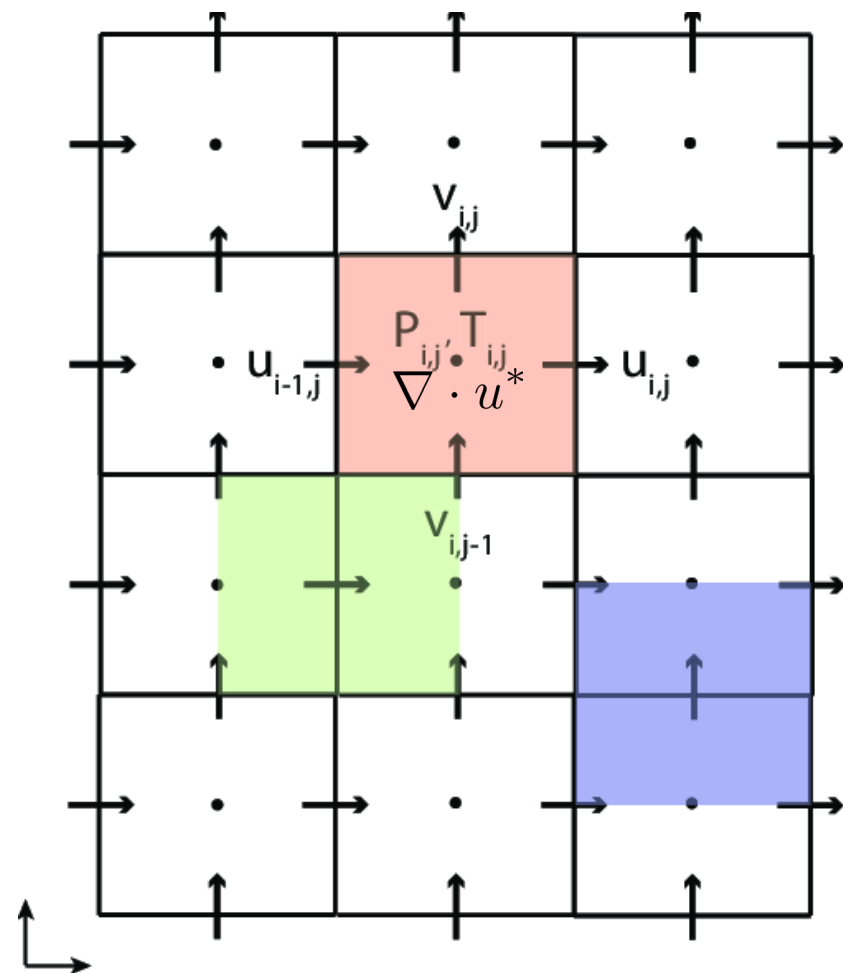
v_sta = np.zeros((nx, nx+1))
v_sta[1:-1, 1:-1] = v_trash

# boundary conditions
v_sta[:,0] = -v_sta[:, 1] # bottom wall
v_sta[:, -1] = -v_sta[:, -2] # top wall
```

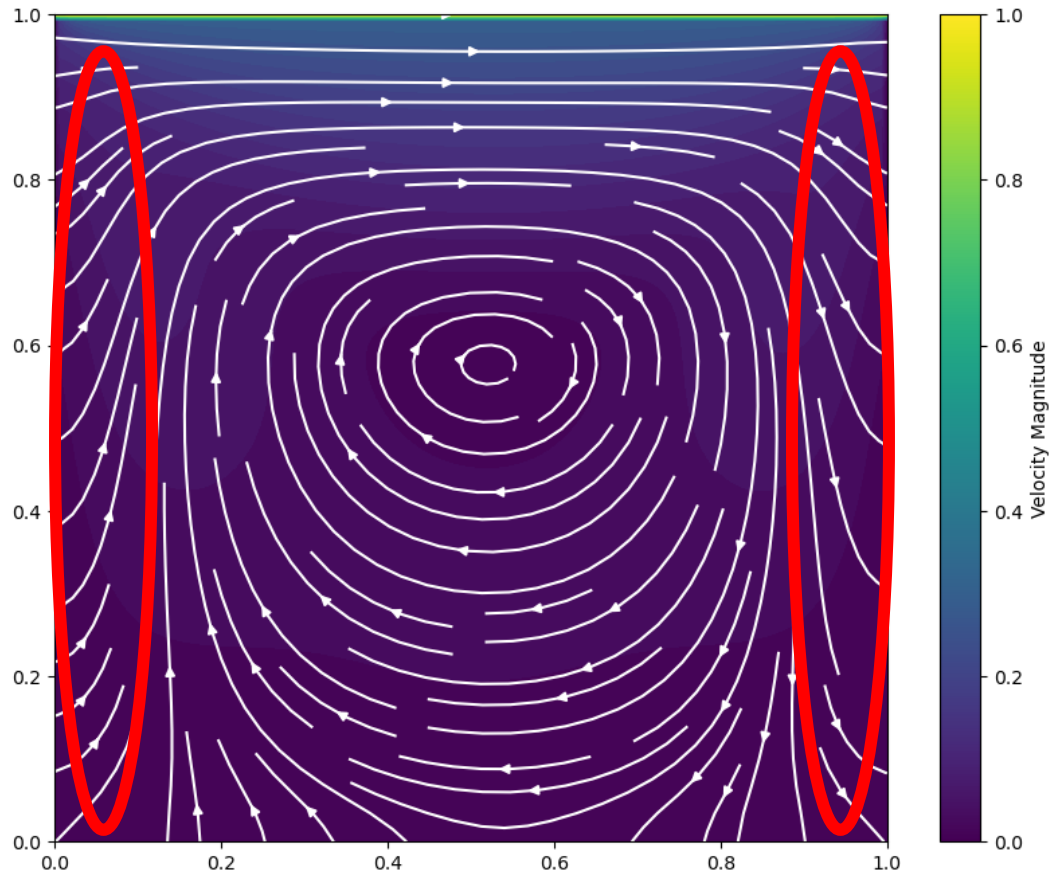
```
# Gauss-Seidel method with successive over-relaxation (SOR)
```

```
def SOR_method(u, h, f, omega=1.9):  
    u_new = np.copy(u)  
    for i in range(1, len(u[0])-1):  
        for j in range(1, len(u[0])-1):  
            u_new[i, j] = (1 - omega) * u[i, j] + omega * (u_new[i+1, j] + u_new[i-1, j] + u_new[i, j+1] + u_new[i, j-1] - h**2 * f[i-1, j-1]) / 4  
    return u_new
```

```
u_sta_dx = (u_sta[1:-1, 1:] - u_sta[1:-1, :-1]) / (dx)  
v_sta_dy = (v_sta[1:, 1:-1] - v_sta[:-1, 1:-1]) / (dx)  
  
del_u = u_sta_dx + v_sta_dy  
  
pi_old = np.zeros((nx+1, nx+1))  
  
pi_new = SOR_method(pi_old, dx, del_u)  
iter = 0  
while np.linalg.norm(pi_old - pi_new) > 1e-5 :  
    iter += 1  
    pi_old = pi_new.copy() # 이전 pi 저장  
    pi_new = SOR_method(pi_old, dx, del_u) # 새로운 pi 계산  
    pi_new[:,0] = pi_new[:,1]  
    pi_new[:,-1] = pi_new[:,-2]  
    pi_new[0,:] = pi_new[1,:]  
    pi_new[-1,:] = pi_new[-2,:]
```



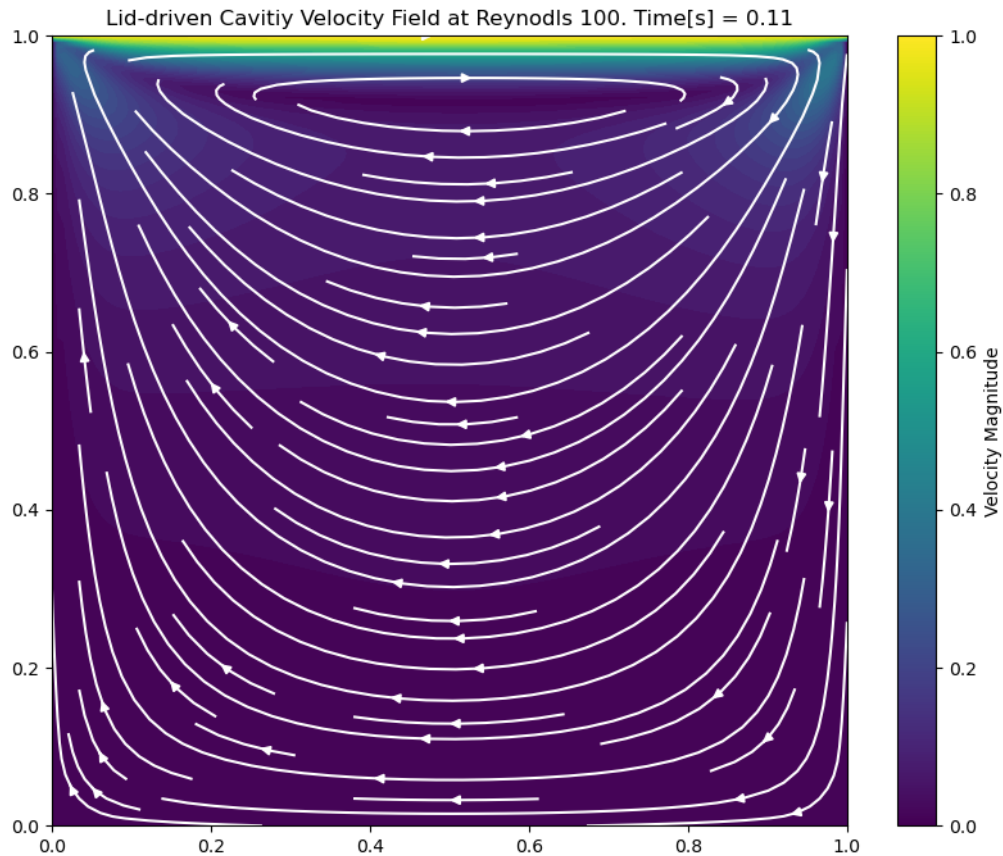

```
while np.linalg.norm(pi_old - pi_new) > 1e-5 :  
    iter += 1  
    pi_old = pi_new.copy() # 이전 pi 저장  
    pi_new = SOR_method(pi_old, dx, del_u) # 새로운 pi 계산
```



```

while np.linalg.norm(pi_old - pi_new) > 1e-5 :
    iter += 1
    pi_old = pi_new.copy() # 이전 pi 저장
    pi_new = SOR_method(pi_old, dx, del_u) # 새로운 pi 계산
    pi_new[:,0] = pi_new[:,1]
    pi_new[:,-1] = pi_new[:,-2]
    pi_new[0,:] = pi_new[1,:]
    pi_new[-1,:] = pi_new[-2,:]

```



[@openfoam homepage](#)

For this case *cavity*, the boundary consists of walls only, split into 2 patches named: (1) *fixedWalls* for the fixed sides and base of the cavity; (2) *movingWall* for the moving top of the cavity. As walls, both are given a *zeroGradient* boundary condition for p , meaning "the normal gradient of pressure is zero". The *frontAndBack* patch represents the front and back planes of the 2D case and therefore must be set as *empty*.

$$\frac{\partial p}{\partial n_{wall}} = 0 \quad \phi = \frac{\Delta t}{\rho} p + C \quad \longrightarrow \quad \frac{\partial \phi}{\partial n_{wall}} = 0$$

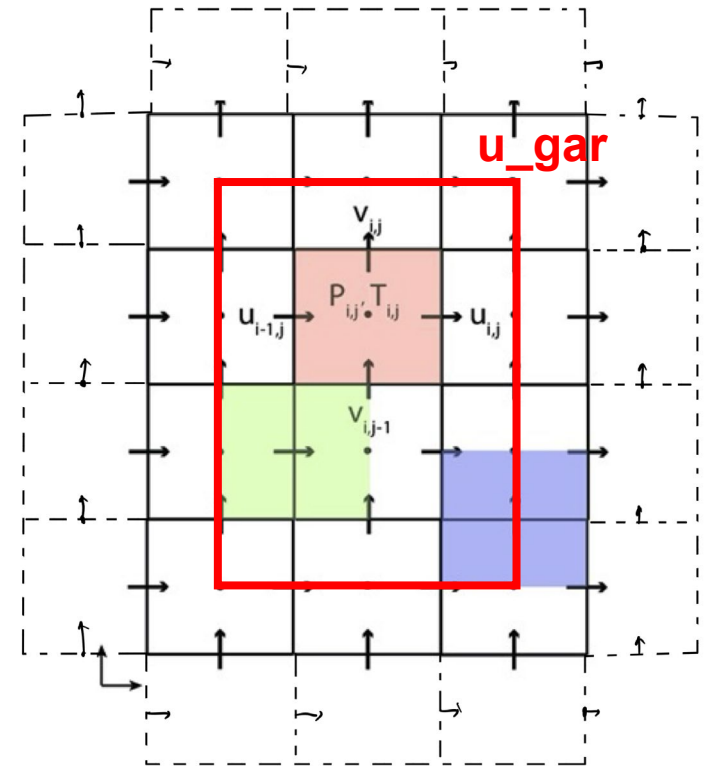
```

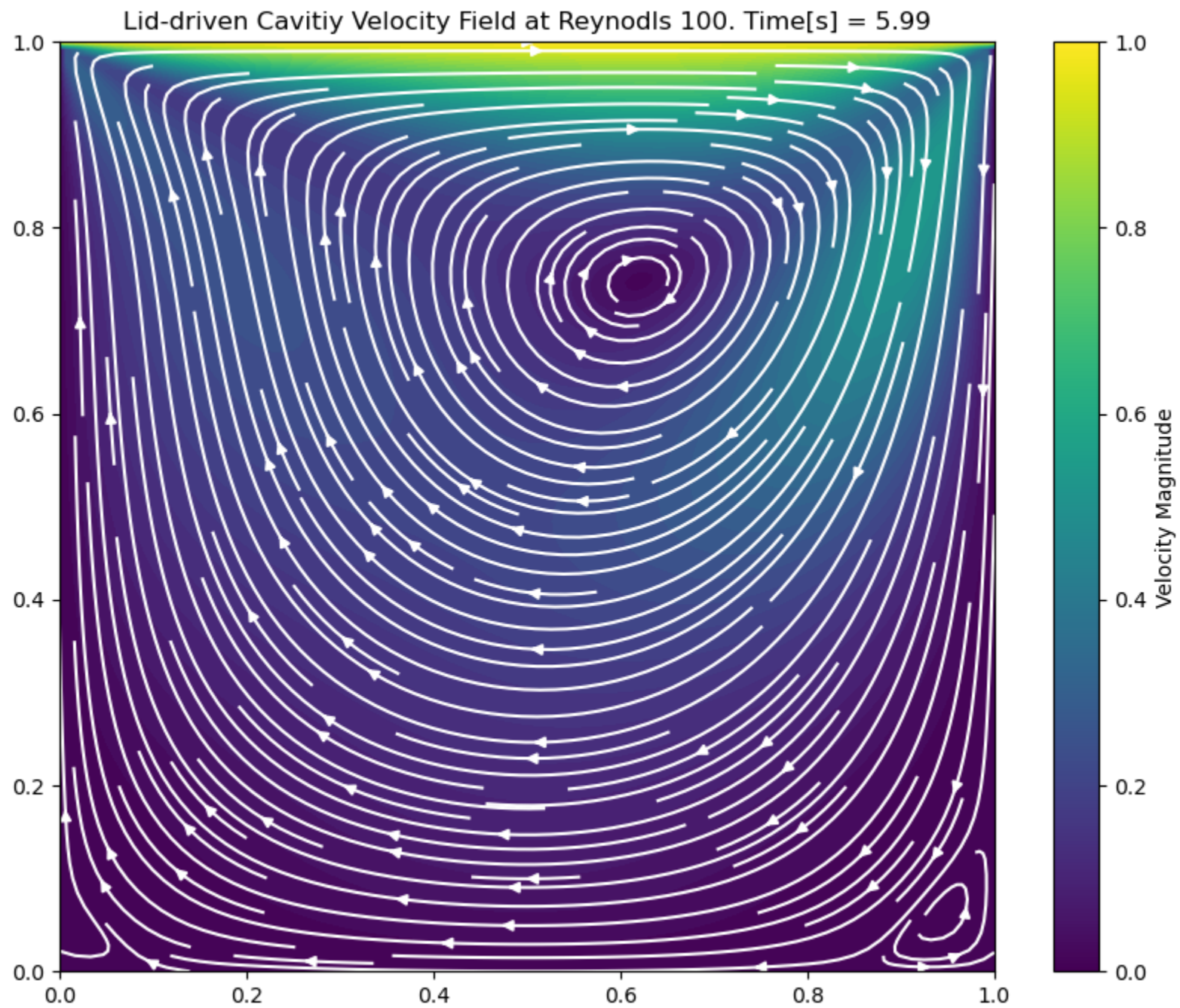
# u_{n+1} 찾기
u_field_new = np.zeros((nx+1, nx))
u_gar = u_sta[1:-1,1:-1] - (pi_new[1:-1,2:-1]-pi_new[1:-1,1:-2])/dx
u_field_new[1:-1,1:-1] = u_gar
u_field_new[0,:] = -u_field_new[1,] # bottom wall
u_field_new[-1,:] = 2-u_field_new[-2,:] # top wall
u_field_list.append(u_field_new)

# v_{n+1} 찾기
v_field_new = np.zeros((nx, nx+1))
v_gar = v_sta[1:-1,1:-1] - (pi_new[2:-1,1:-1]-pi_new[1:-2,1:-1])/dx
v_field_new[1:-1,1:-1] = v_gar
v_field_new[:,0] = -v_field_new[:,1] # left wall
v_field_new[:, -1] = -v_field_new[:, -2] # right wall
v_field_list.append(v_field_new)

```

$$\mathbf{u}^* = \mathbf{u}^{n+1} + \nabla \phi$$





Q&A